

LANGUAGE MODELLING FOR LEARNING ROBUST POLYMER REPRESENTATIONS

A REPORT

Submitted by

RAHUL KRISHNA K

CH19B022

in partial fulfilment of requirements for

MASTER OF TECHNOLOGY



DEPARTMENT OF CHEMICAL ENGINEERING

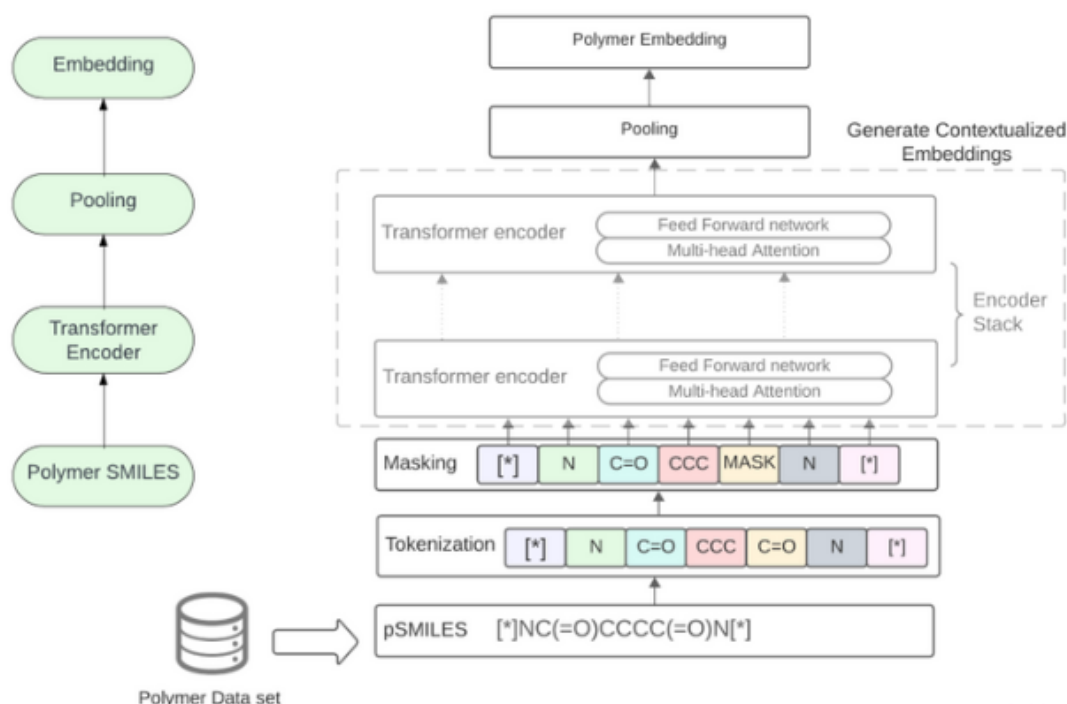
INDIAN INSTITUTE OF TECHNOLOGY MADRAS

CHENNAI-600 036

MAY 2024

Abstract

Machine Learning frameworks have enabled substantial progress in the development of polymer property predictors. One of the vital stages in the Polymer Informatics pipeline is the conversion of the chemical molecules usually indexed as textual representations called "SMILES" into numerical features that can be easily interpreted by machine learning models. The accuracy of these models heavily depends on the quality of feature space representations. Scarcity of property data in the synthesized space of polymers makes data driven research in Polymer Informatics challenging and raises the need to focus on learning better representations. Our focus is on building systems that learn representations from millions of examples of SMILES opposed to handcrafting rules to generate it. We present a language modeling approach to learn more unique and robust representations, or embeddings to enable machine learning models to make better predictions. In this pipeline we generate machine-crafted embeddings from polymer SMILES treating it as a "chemical language" and our language model generating it as a "chemical linguist" which inherently learns from millions of polymer sequence data. We used encoder blocks from transformer coupled with masked language modelling to learn the SMILES language and extract features. SMILES is relatively a simple language but building special vocabulary improved the model performance compared to character level language model. The present approach outperforms all traditional hand crafted fingerprint schema in accuracy of property prediction and uniqueness in low dimensional projections making this a strong candidate for accelerating polymer research. Recent advances in language modeling and AI foundation models can be leveraged to improve model architectures.



Colophon

This paper is typeset with $\text{\LaTeX}$. Canva was used to generate some of the figures.

Code, $\text{\LaTeX}$, and Data Availability

Code available for academic use at: BERT-PolyLang. The Dataset for pre-training: polyOne Data Set, ZINC-15 database.

Acknowledgements

First and foremost, I would like to express my sincere gratitude to my thesis guide, Dr. Tarak Kumar Patra, who has graciously offered technical feedback support, guidance, and mentorship throughout this journey.

I am deeply indebted to my friends, Danish, Anfas, Juraij, Jithin, Manu, Sreejith, Nohan, Rinkle, Veena and Dhanya for their constant support, encouragement, and unwavering belief in me. Their presence has been a source of motivation and strength during the most challenging times.

I would also like to extend my heartfelt thanks to my family, especially my parents, Radhakrishnan K ,Vineetha K and my sister Raveena K for their unconditional love, understanding, and mental support.

Furthermore, I would like to express my sincere gratitude to Andrej Karpathy, a renowned AI practitioner, for his detailed and valuable tutorials. His insights and expertise have been instrumental in guiding me through the intricate realms of artificial intelligence and machine learning, laying the foundation for this work.

I would also like to thank all PRGian(my lab mates) for their support and feed backs during lab meetings. Their encouragement and assistance have been invaluable in shaping this thesis into its present form.

Contents

1	Introduction	5
1.1	Thesis Objectives	6
2	Critical Review of the Literature	7
3	SMILES: Text Representation of Molecules	9
3.1	SMILES Specification Rules	10
3.1.1	Atoms	10
3.1.2	Bonds	10
3.1.3	Branches	11
3.1.4	Cyclic Structures and Aromaticity	11
3.1.5	Stereo chemistry	12
3.2	Extensions	12
4	Existing Finger Printing Approaches	14
4.1	One-Hot Encoded Fingerprints	15
4.2	RdKit Physico-Chemical Descriptors as Fingerprints	16
4.3	Morgan Fingerprints	17
4.4	QR-code based Fingerprints	17
4.5	UMAP plot of Molecular Fingerprints	18
5	Research Motivation	20
5.1	What are embedding	20
5.2	Analogy to Language Modelling	20
5.3	Scope of Encoder-Decoder Models	21
6	Modeling	22
6.1	Input Representations: Tokenization	22
6.2	Building Vocabulary	22
6.3	Transformer Architecture	26
6.3.1	Attention	26
6.3.2	Scaled Dot-Product Attention	27
6.3.3	Single Head of Attention	27
6.3.4	Multi-Head Attention	28
6.3.5	Position-wise Feed-Forward Networks	29
6.3.6	Encoder Block	30
6.3.7	Embedding and Softmax	31
6.3.8	Positional Encoding	31
6.4	Additional resources	31
6.5	Language Model	32
6.6	Sentence-Transformer	33
6.7	Masked Language Modelling	33
6.8	Pre-Traning using Masked Language Modelling	34
7	Results and Discussions	36
8	Conclusions and Future Work	40

1 Introduction

Language modeling and in-general Generative AI are rapidly increasing in popularity, demand, and potency. With innovative data-driven approaches vastly reshaping the realm of polymer informatics(PI), we wish to push the frontiers by incorporating modern language modeling recipes inspired by Natural Language Processing in polymer property predictions. Polymers are an integral part of our day-to-day life and play a pivotal role in the chemical and pharmaceutical industry. Their chemical space is so diverse and large making it challenging for designing or choosing application-specific candidates at the same time unleashes tremendous potential for material discoveries. Several Machine learning(ML) frameworks have been developed for molecular property prediction of polymers from chemical structures and representations. One of the major challenges in the PI pipeline is conversion of the chemical structures into numerical representations that can be easily interpreted by ML models. These conversions are often called fingerprinting, descriptor or feature extraction. Traditionally, fingerprinting techniques are hand-crafted which utilizes chem informatics tools to numerically embed key structural and functional features. Although quite effective these methods require an in-depth understanding of the rule-based methods of conversion and lack generalization. One of the problems faced in material research is the limited availability of synthesized polymers data and often requires extrapolation of new polymer chemical classes.

Handcrafted methods like Morgan fingerprinting [30] , Motif-based fingerprinting [12] , Rdkit descriptors are prone to error during extrapolation and require additional enhancements. To mitigate this issue we are exploring fully-machine crafted fingerprints replacing handcrafted methods. Every polymer in its monomeric form can be represented as a unique canonicalized SMILES [37] (Simplified Molecular-Input Line-Entry System) string. In this work we explore machine-crafted fingerprinting from polymer SMILES(pSMILES) treating it as a **“chemical language”** and our language model generating the fingerprint as a **“chemical-linguist”** which inherently learns from millions of polymer sequence data. We also have experimented on fingerprinting polymer SMILES data on a QR(Quick Response)[34] code which also serves as an additional machine-crafted fingerprint for bench marking. Generating QR codes is effortless and doesn’t require any specific domain knowledge but still provides outstanding results comparable to that of traditional handcrafted methods. The main focus of the project is to produce a robust embedding leveraging transformer-based architecture[35] when paired with a state-of-the-art machine learning model yields better property predictions.

1.1 Thesis Objectives

- **Objective 1** - Build language model trained on polymer SMILES to generate embedding for polymers and use it for property predictions
- **Objective 2** - Compare the quality of embeddings to that of traditional molecular finger printing methods such as One Hot Encoding, Morgan fingerprint for property prediction task.
- **Objective 3** -Visualise the embedding latent space of different finger prints using low dimensional projections.

2 Critical Review of the Literature

Development of the SMILES chemical notation system was initiated by D. Weininger [37] in 1987. Although it could represent all the small organic molecules, rules to represent polymers was unclear due to the larger number of monomeric units and different monomeric sites. Different rules apply for encoding atoms, bonds, branches, cyclic structures, disconnected structures and aromaticity. While substantial advances have been made for small molecules, the polymer community has struggled in coming up with an efficient representation system. In order to represent polymers efficiently, extensions of SMILES are being used in Polymer Informatics such as the Polymer Genome project [18] which apply rules to encode recurring unit of homo-polymers with [*] to represent the monomeric attachment site. Recently, researchers from MIT also came up with a new representation system (chemical notation based on SMILES) called BigSMILES[20] that is capable of handling the stochastic nature of polymers overcoming the difficulty that limits the applicability of all deterministic representations developed for small molecules.

A major limitation in material science applications especially in polymer informatics is the lack of readily available property data[2]. Several polymer databases and/or platforms, including PolyInfo[27], Polymer Genome[18], and CHEMnetBASE-Polymers, have been created, and research of polymer informatics based on these databases have been carried out. Most of these polymer databases are embedded in web-based applications and are not available on a large scale. One initiative to enable open source large scale polymer informatics was the creation of PI1M 10[22] (referring to 1 million polymers for polymer informatics). PI1M 10 uses a generative model which is trained on 12 000 polymers manually collected from the largest existing polymer database PolyInfo, and then the model is used to generate 1 million polymers.

SMILES representation is a form of line notation using ASCII string and are text representations. In order for the ML frameworks to process it we need to convert it into a numerical representation often referred to as "Fingerprints", "descriptors or feature extraction". There are many ways to numerically encode information contained in a SMILES string. One such method is a simple one-hot-encoding. An extension of this simple method implemented in the work SMILES2Vec[9] is proven to be very effective for Deep Neural Networks which inherently learns from SMILES one hot representation without the need for additional feature engineering. In 2015, Rampi Ramprasad et al.[12] have experimented with a class of hierarchical motif-based topological fingerprints for materials composed of elements such as C, O, H, N, F, etc., whose coordination preferences are well understood. Another method which captures the topological features using circular atom neighborhoods is called Morgan Fingerprinting[30]. Hirohara, Maya, et al.[11] have recently developed a pipeline using CNN based methods on morgan fingerprints.

These classes of methods can be classified as handcrafted fingerprints and they are often built on top of invaluable intuition, experience and in-depth domain knowledge. One key limitation with the handcrafted methods is lack of generalization, ML pipelines that use handcrafted fingerprints are prone to errors during extrapolation of a new polymer chemical class. To address the above barriers, Kuenneth and Ramprasad replaced handcrafted finger print with a NLP inspired Transformer fingerprint called PolyBERT[19] for predicting polymer properties. Proceeding from Google's Attention is all you need paper[35], transformers have gained a lot of popularity and swiftly became the gold standard in ML language modeling. Earlier experiments of using transformer based architecture for property prediction started with a simple BERT[7] model by Wang et al.

SMILES-BERT[36] model is a semi-supervised approach which consists of an attention mechanism based transformer layer. The proposed method used BERT’s latent space as molecular fingerprint and outperformed former state-of-the-art models including the unsupervised recurrent neural network and graph neural networks. A very recent study by Xu et al.[39] used a RoBERTa[21] model (an evolution of the BERT 14 Transformer model) for polymer property predictions. Although this RoBERTa model pre-trained on 5 million polymer SMILES performed excellently well, this study does not include any comparison with the state-of-the-art handcrafted fingerprint. Around the same time, PolyBERT[19] used a DeBERTa[10] model and was pretrained on 100 million pSMILES outperformed every existing transformer based polymer property prediction model. After careful consideration of the work around PolyBERT we note that the sampling of 100 million pSMILES was done randomly from (Breaking Retro Synthetically Interesting Chemical Substructures)BRICS[6] decomposition of the 13000 synthesized polymers. PolyBERT uses a encoder only model and doesn’t aim to solve the inverse problem which till date remains majorly unexplored. One of the few attempts to inverse problem by creating a informative and clustered latent space was achieved by MegaMolBART[26] by NVIDIA CLARA. Extending to a encoder-decoder model could open up new possibilities for solving the inverse problem of generating the structure from a property data. NVIDIA CLARA team has also experimented this using Mutual Information Machine MIM[29].

Another limitation of polyBERT is that it is non-interpretable for copolymers as it is invariant to the order of monomers. One major challenge when deriving language modeling concepts is that everything that works well for English language not necessarily works for SMILES chemical language as they can be considered more of an unnatural language as opposed to natural language. Hand crafted fingerprints often suffer from the lack of generalization[3] when extrapolating to new chemical classes. The problem with generalization can be solved with sufficiently large amounts of property data, but they are not readily available for polymers. Modern research explores an alternative route of better representing these chemical spaces before mapping it with properties.

3 SMILES: Text Representation of Molecules

Introduced in 1987 by David Weininger [37], SMILES which stands for "Simplified Molecular Input Line Entry System" is a text based chemical notation designed for chemical information processing. SMILES is a chemical notation system designed for modern chemical information processing. Based on principles of molecular graph theory, it allows rigorous structure specification using a concise and natural grammar. SMILES is a chemical notation language specifically designed for computer use by chemists, making it easily accessible yet flexible enough to enable interpretation and generation of chemical notation independent of the specific computer system in use. Similar to conventional chemical notation, SMILES improves on conventional software methods by offering greater speed and better utilization of computer capacity, enabling unique and accurate specification of molecular structures suitable for use with chemical databases.

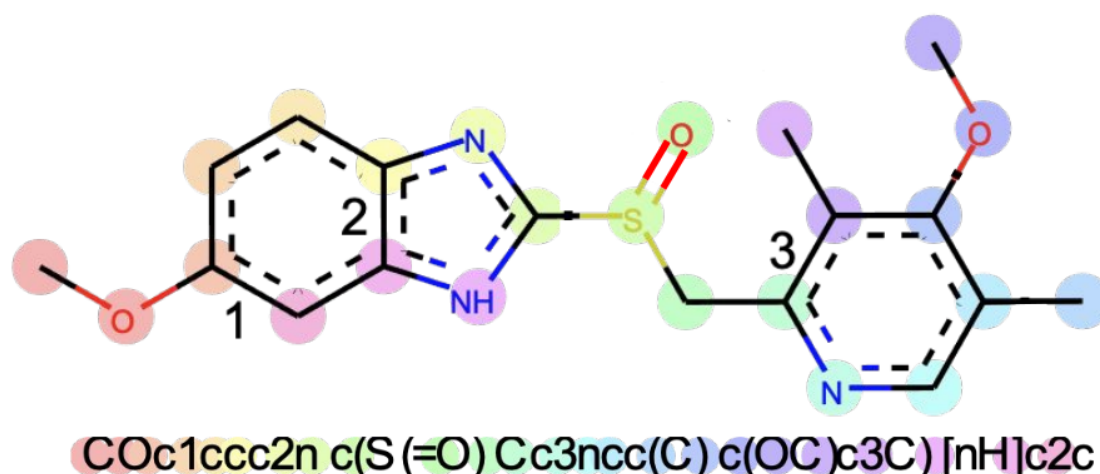


Figure 1: Example of a SMILES string and their chemical 2D structure

SMILES contains the same information as an extended connection table but is more useful as it is a "linguistic construct" rather than a computer data structure. It is a relatively simple language with a basic vocabulary of atoms and bond symbols, along with a few grammar rules. The vocabulary can be extended to incorporate functional groups or other chemically meaningful fragments, treating them as "words" for storing chemical information and enabling chemical intelligence. This linguistic nature allows SMILES to capture the essence of chemical entities with distinguished properties.

One of the key advantages of SMILES is its compact representation compared to other methods. SMILES strings occupy approximately 50-70% less space than equivalent connection tables, even in binary form. By defining an extended vocabulary and encoding these strings as numerical representations using simple lookup tables, further compression can be achieved. This compact and unique representation not only captures the molecular physics and chemistry of structures but also opens endless possibilities for chemical information programmers. Examples of SMILES applications include keys for database access, mechanisms for researchers to exchange chemical information, entry systems for chemical data, and components of languages for artificial intelligence or expert systems in chemistry.

"SMILES enables small molecules and polymer monomers to be represented as text leverage deep learning advancements in Natural Language processing"

3.1 SMILES Specification Rules

SMILES notation consists of a series of characters containing no spaces. Hydrogen atoms may be omitted (hydrogen-suppressed graphs) or included (hydrogen-complete graphs). Aromatic structures may be specified directly or in Kekulé form. There are five generic SMILES encoding rules [14], corresponding to specification of atoms, bonds, branches, ring closures, and disconnections.

3.1.1 Atoms

Atoms are represented by their atomic symbols: this is the only required use of letters in SMILES. Each non-hydrogen atom is specified independently by its atomic symbol enclosed in square brackets, []. The second letter of two-character symbols must be entered in lower case. Elements in the "organic subset" B, C, N, O, P, S, F, Cl, Br, and I may be written without brackets if the number of attached hydrogens conforms to the lowest normal valence consistent with explicit bonds. "Lowest normal valences" are B (3), C (4), N (3,5), O (2), P (3,5), S (2,4,6), and 1 for the halogens. Atoms in aromatic rings are specified by lower case letters, e.g., aliphatic carbon is represented by the capital letter C, aromatic carbon by lower case c. Since attached hydrogens are implied in the absence of brackets, the following atomic symbols are valid SMILES notations. Atoms with valences other than "normal" and elements not in the "organic subset" must be described in brackets. Within brackets, any attached hydrogens and formal charges must always be specified.

Table 1: *Atoms Represented as SMILES Notations*

SMILES	Chemical_name	Chemical_formula
C	methane	(CH ₄)
P	phosphine	(PH ₃)
N	ammonia	(NH ₃)
O	water	(H ₂ O)
[S]	elemental sulfur	S
[Au]	elemental gold	Au
[H+]	proton	H ⁺
[OH-]	hydroxyl anion	OH ⁻

3.1.2 Bonds

Single, double, triple, and aromatic bonds are represented by the symbols -, =, #, and :, respectively. Adjacent atoms are assumed to be connected to each other by a single or aromatic bond (single and aromatic bonds may always be omitted). Examples are:

Table 2: Bonds Represented in SMILES Notations

SMILES	Chemical_name	Chemical_formula
CC	ethane	(CH ₃ CH ₃)
C=O	formaldehyde	(CH ₂ O)
C=C	ethene	(CH ₂ =CH ₂)
O=C=O	carbondioxide	(CO ₂)
C#N	hydrogen cyanide	(HCN)
[H][H]	molecular hydrogen	H ₂

3.1.3 Branches

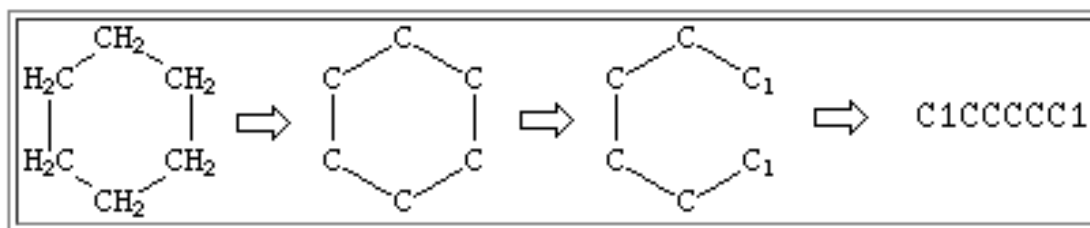
Branches are specified by enclosing them in parentheses, and can be nested or stacked. In all cases, the implicit connection to a parenthesized expression (a "branch") is to the left. Examples are:

Table 3: Branches Represented in SMILES Notations

SMILES	Chemical_name
CCN(CC)CC	Triethylamine
CC(C)C(=O)O	Isobutyric acid
C=CC(CCC)C(C(C)C)CCC	3-propyl-4-isopropyl-1-heptene

3.1.4 Cyclic Structures and Aromaticity

Cyclic structures are represented by breaking one bond in each ring. The bonds are numbered in any order, designating ring opening (or ring closure) bonds by a digit immediately following the atomic symbol at each ring closure. This leaves a connected non-cyclic graph which is written as a non-cyclic structure using the three rules described above. Cyclohexane is a typical example:

**Figure 2:** Example of Cyclohexane SMILES string and its chemical 2D structure

Aromaticity must be deduced in a system such as SMILES which generates an unambiguous chemical nomenclature because of the fundamental requirement to characterize the symmetry of a molecule. Given effective aromaticity-detection algorithms, it is not necessary to enter any structure as aromatic if the user prefers to enter an aliphatic (Kekulé-like) structure. Entering structures as aromatic directly (i.e., by using lower case atomic symbols) provides a shortcut to accurate chemical specification and is closer to the mental molecular model used by most chemists.

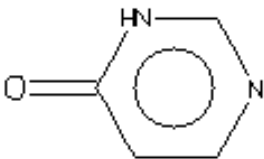
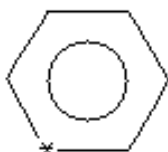
		
<chem>C1=COC=C1</chem>	<chem>C1=CN=C[NH]C(=O)1</chem>	<chem>C1=C*=CC=C1</chem>
<chem>c1cocc1</chem>	<chem>c1cnc[nH]c(=O)1</chem>	<chem>c1c*ccc1</chem>

Figure 3: Example of aromatic compound's SMILES string and its chemical 2D structure

3.1.5 Stereo chemistry

SMILES permits, but does not require, specification of stereoisomers. Configuration around double bonds is specified using the characters / and \ to show directional single bonds adjacent to a double bond. For example, F/C=C/F is one representation of trans-1,2-difluoroethylene, in which the fluorine atoms are on opposite sides of the double bond, whereas F/C=C\F is one possible representation of cis-1,2-difluoroethylene, in which the fluorines are on the same side of the double bond.

Bond direction symbols always come in groups of at least two, of which the first is arbitrary. That is, F\C=C\F is the same as F/C=C/F. When alternating single-double bonds are present, the groups are larger than two, with the middle directional symbols being adjacent to two double bonds. For example, the common form of (2,4)-hexadiene is written C/C=C/C=C/C.

3.2 Extensions

SMILES was one of the earliest attempts to create text based representations. Following this, **SMARTS** [13] is a line notation for specification of sub structural patterns in molecules. While it uses many of the same symbols as SMILES, it also allows specification of wildcard atoms and bonds, which can be used to define sub structural queries for chemical database searching. One common misconception is that SMARTS-based sub structural searching involves matching of SMILES and SMARTS strings. In fact, both SMILES and SMARTS strings are first converted to internal graph representations which are searched for sub graph isomorphism.

While substantial advances have been made for small molecules, the polymer community has struggled in coming up with an efficient representation system. In order to represent polymers efficiently, extensions of SMILES called **pSMILES** are being used in Polymer Informatics such as the Polymer Genome project [18] which apply rules to encode recurring unit of homo-polymers with [*] to represent the monomeric attachment site. As the SMILES format described here is custom-designed by us for polymers, it is not completely identical to other SMILES formats. Atoms other than the first and last can also be assigned as the linking atoms by adding special symbol, [*]. As an example, C(C=C1)=CC=C1 represents poly(p-phenylene) with link through para positions, while [*]C(C=C1)=CC([*])=C1 and C(C=C1)=C([*])C=C1 have connecting positions at meta and ortho positions, respectively.

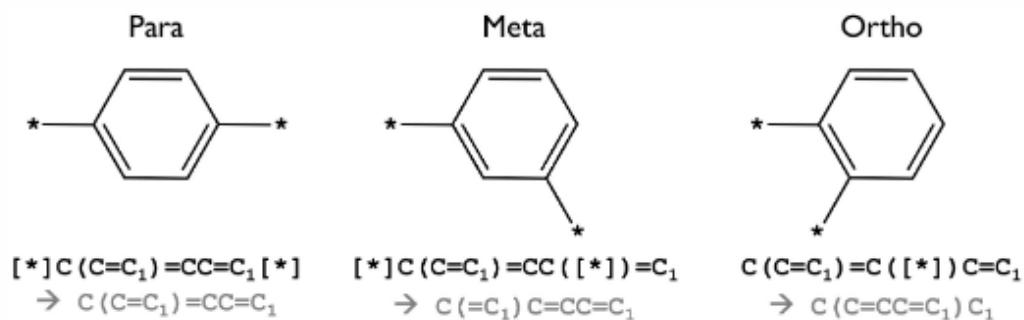


Figure 4: Example of SMILES using[*] to specify a connection point. Equivalent SMILES without [*] is shown together. C(C=C1)=CC=C1 represents poly(*p*-phenylene) with link through para positions, while [*]C(C=C1)=CC([*])=C1 and C(C=C1)=C([*])C=C1 have connecting positions at meta and ortho positions, respectively. “Adapted from the Polymer Genome Project Guide, available at

4 Existing Finger Printing Approaches

Molecular fingerprints, or simply fingerprints, are numerical representations of molecules that play a crucial role in various machine learning applications involving chemical structures. As SMILES (Simplified Molecular Input Line Entry System) are text-based representations, they need to be processed through an encoding schema to convert them into numerical representations suitable for machine learning algorithms and downstream prediction tasks.

These encoding schema can range from simple approaches, such as mapping characters to integers using a lookup table, to more complex methods, like employing attention mechanisms to embed patterns, as in our work. There have been several approaches to generating molecular fingerprints, which can be broadly classified into two categories: handcrafted and machine-crafted.

Handcrafted fingerprints rely on invaluable intuition, experience, and tedious development processes, often involving complex computations that can consume a significant portion of time during model training and inference. However, these handcrafted fingerprints may lack generalization capabilities [19], as hand-crafted rules can have exceptions, particularly when dealing with potential molecules outside the synthesized space, a common scenario in drug discovery.

On the other hand, machine-crafted fingerprints are encoding schema developed by machines through exposure to a large number of SMILES examples. These learned representations, rather than hard-coded rules derived from the synthesized space, offer the potential for improved generalization and extrapolation capabilities compared to handcrafted fingerprints.

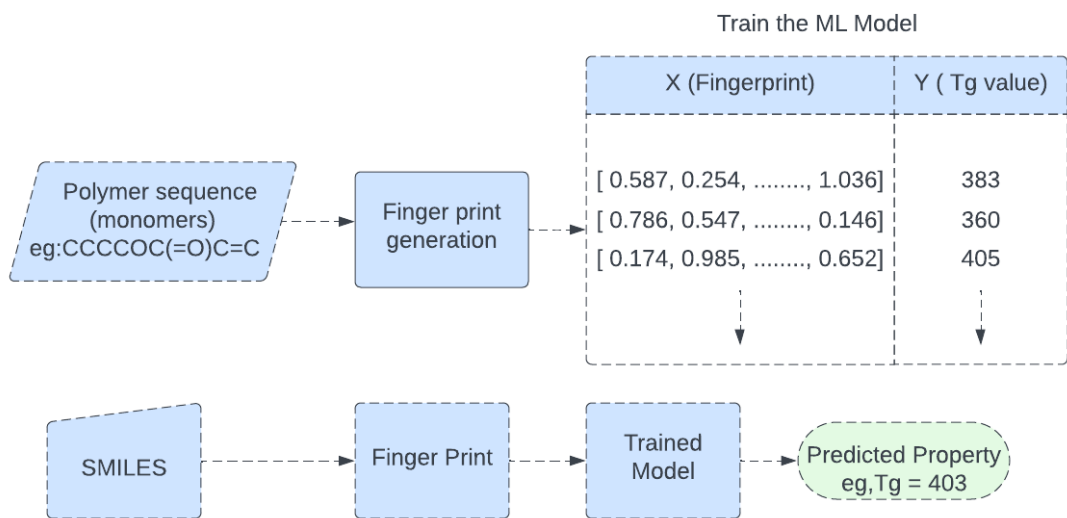


Figure: Workflow for using ML for property prediction

Figure 5: ML Workflow for property prediction using fingerprints

Fingerprints are essential for machine learning applications involving chemical structures. Their unique ability to represent molecules in a higher-dimensional latent space is vital for machine learning models to distinguish molecules based on their properties

and leverage this informative, clustered latent space to predict the properties of unseen molecules with high accuracy. The choice between handcrafted and machine-crafted fingerprints may depend on the specific requirements of the application, the available computational resources, and the desired balance between interpretability and generalization performance.

4.1 One-Hot Encoded Fingerprints

One-hot encoding is a technique for converting categorical data into numerical data by assigning a binary vector to each category. In the context of SMILES (Simplified Molecular Input Line Entry System) strings, which are linear representations of molecular structures, one-hot encoding can be performed at the character level or the atom level.

RDKit, a library for cheminformatics, facilitates reading, drawing, and analyzing molecules from SMILES strings. One-hot encoding at the atom level represents each atom in a molecule as a binary vector, indicating its presence or absence in a predefined set of atom types. For instance, if the set of atom types is C, N, O, H, the one-hot encoding for C would be [1, 0, 0, 0], for N would be [0, 1, 0, 0], for O would be [0, 0, 1, 0], and for H would be [0, 0, 0, 1]. This approach allows for the unique identification of each atom by its corresponding one-hot vector.

One-hot encoding at the atom level can be advantageous for molecular representation in graph form, where each node corresponds to an atom, and each edge corresponds to a bond. The one-hot vectors can serve as node attributes, capturing atomic properties. Alternatively, one-hot encoding at the atom level can be applied to SMILES strings by replacing each character with its corresponding one-hot vector. The `MolFromSmiles` function from RDKit converts each SMILES string into a molecule object, while the `GetAtoms` function retrieves a list of atoms for each molecule. The `GetSymbol` function obtains the symbol for each atom, and the NumPy library facilitates finding an index for each symbol in the set of atom types, followed by creating an identity matrix with the same size as the set of atom types, and obtaining the one-hot encoded vector for each index from the identity matrix.

An alternative approach to using RDKit for implementation is presented by **SMILES2Vec** [9]. This method is an extension of the popular one-hot encoding method. Initially, the maximum length of the encoding is set 5 units above the length of the longest SMILES sequence. Subsequently, all SMILES are standardized to the same length by padding with zeros. Later, using a character-to-index map, an encoding matrix is created with binary values, "1" for the presence of a character and "0" for its absence. This 2D vector serves as a one-hot-encoded fingerprint.

A typical workflow for using one-Hot encoding as binary image input [3] for Convolutional Neural Networks to predict glass transition temperature of polymers is shown below in Figure

While molecular descriptor vectors may not always match the predictive performance of more sophisticated molecular representation methods, such as extended-connectivity fingerprints or graph neural networks, they offer the advantage of interpretability. Many of the computed descriptors have well-established chemical and physical meanings, allowing for a more intuitive understanding of the molecular properties by chemical experts.

Popular Chem Informatics tool RDKit provides 209 physico-chemical descriptors. This 209 long linear 1D vector serves as a fingerprint. To generate RDKit Descriptors, one can use `getMolDescriptors()` function with a mol object as input.

4.3 Morgan Fingerprints

Morgan (Circular) Fingerprinting is a technique for generating molecular fingerprints that captures the structural features of molecules by exploiting circular atom neighborhoods. The RDKit, an open-source cheminformatics toolkit, supports various fingerprinting algorithms, including Morgan Fingerprints, through its `rdFingerprintGenerator` module.

The generation of Morgan Fingerprints commences with the assignment of initial atom identifiers based on properties derived from the Daylight atomic invariant rule, considering factors such as atomic number, degree, number of attached hydrogens, and formal charge. Subsequently, for each radius r ranging from 0 to a maximum value (typically 2 or 3), the identifiers of all atoms within r bonds of each atom are combined using a hash function to generate a new identifier for that atom. This process captures the circular substructures surrounding each atom, and the resulting set of identifiers for each radius can be converted into a bit vector by folding and hashing or alternatively used as counts to generate a count vector.

The final Morgan fingerprint is the union of all circular substructures for all radii, encompassing both local and global structural features of the molecule. This approach allows for efficient computation and comparison of molecular fingerprints, making it a valuable tool for similarity searching, clustering, and virtual screening in chem informatics applications.

To generate Morgan Fingerprints using RDKit, one can create a `MorganGenerator` object with the desired radius and fingerprint size parameters. The `GetFingerprint` method can be used to obtain a bit vector of the specified size, while the `GetSparseFingerprint` method yields a sparse bit vector representation. Alternatively, the `GetCountFingerprint` or `GetSparseCountFingerprint` methods can be employed to generate count vectors instead of bit vectors. It is noteworthy that the Morgan Fingerprints generated by RDKit are roughly equivalent to the ECFP/FCFP fingerprints used in other software, with the radius parameter in RDKit corresponding to half of the diameter parameter in ECFP/FCFP.

4.4 QR-code based Fingerprints

QR code fingerprinting [34] is an innovative approach to representing molecular structures as two-dimensional barcodes, which can be utilized as input for machine learning models, such as convolutional neural networks (CNNs), for predicting material properties. This technique leverages the byte mode encoding scheme, where ASCII characters are represented as strings of eight binary digits or bytes. In the QR code generation process, the binary data is segmented into frames, commonly referred to as "data code words." These code words undergo Reed-Solomon error correction, and the resulting combination of code words and error correction code words is embedded into the QR code. The `pyqrcode`

module in Python can be employed to facilitate the generation of QR codes from molecular data.

The QR code fingerprinting approach offers a unique representation of molecular structures, enabling the application of CNNs for predicting material properties. By encoding molecular information into two-dimensional bar codes, this technique provides a novel way to leverage the power of CNNs in the field of chem informatics and material science.

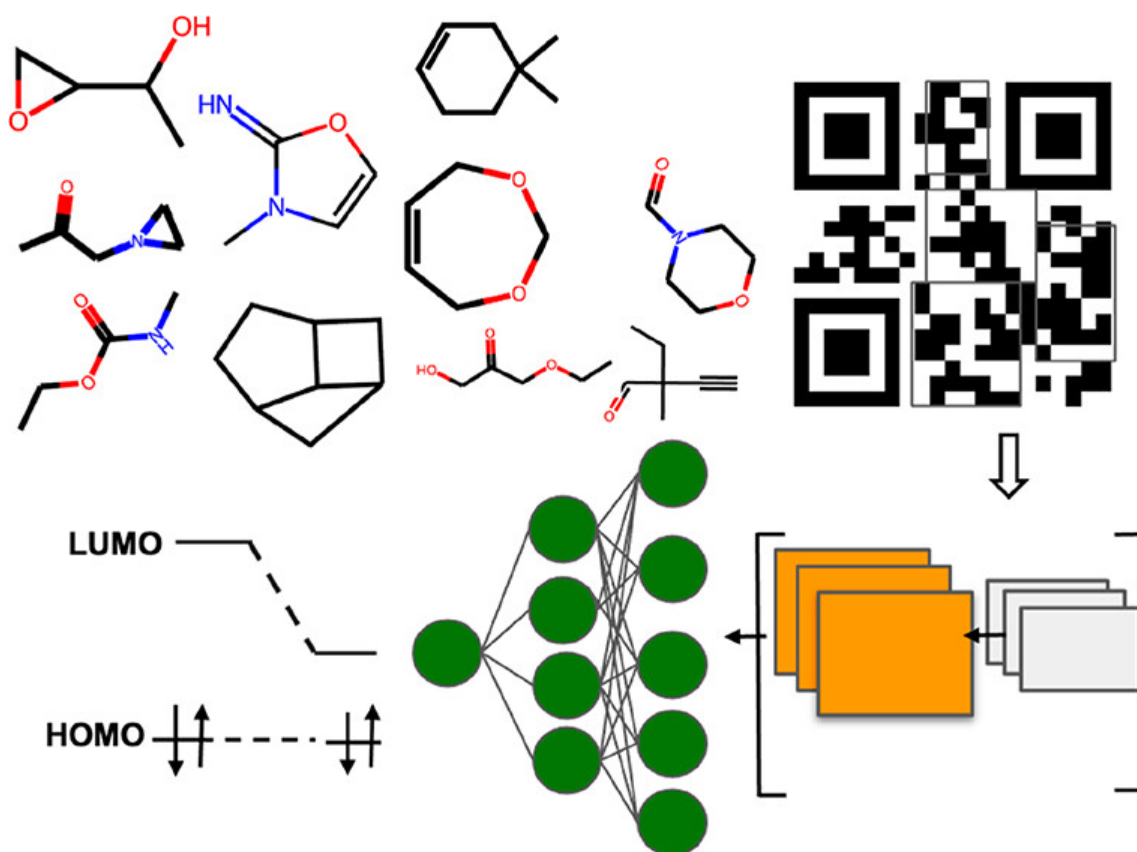


Figure 7: The 2D UMAP plot of molecular fingerprints. The One Hot Encoding, Morgan Fingerprint and QR Code Fingerprint of 21000 molecules from QM 9 24 Dataset are mapped to 2D latent space in (a), (b), and (c), respectively. The color bar is adjusted according to their specific heat values. available at

4.5 UMAP plot of Molecular Fingerprints

To better understand how different fingerprint representations span the chemical space we performed Uniform Manifold Approximation and Projection, UMAP. UMAP is a non linear dimensionality reduction technique which allows the visualization of higher dimensional data in two or three dimensional space. The data we used for the plotting is of small organic molecules collected from QM9 24 dataset. From figure 7 it is evident that QR code fingerprint forms distinctive clusters and longer inter point distance in 2D space. We argue that molecules in the same cluster have identical topological features.

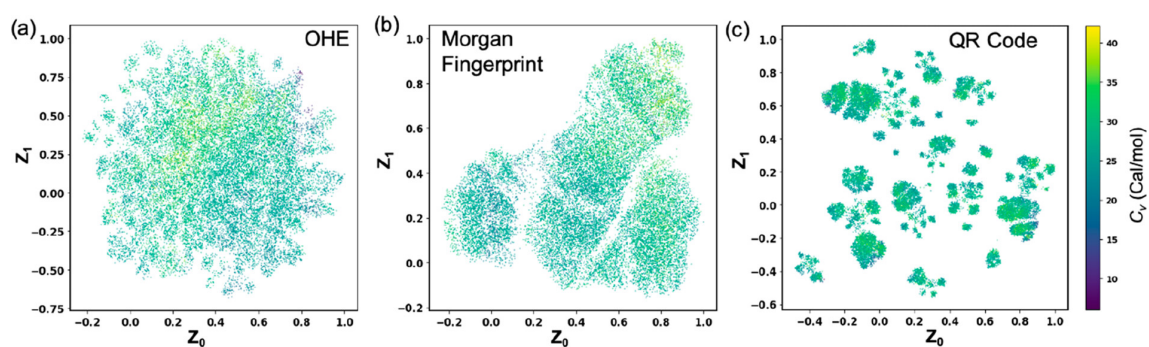


Figure 8: Workflow of QRChEM. The SMILES representations of molecules are converted to QR codes. The QR codes of molecules are fed to the CNN, which consists of convolution layers, pooling layers, and deep layers. The last layer of the deep layers produces the properties of the molecules available at

5 Research Motivation

5.1 What are embedding

Embeddings have emerged as a crucial component in the implementation of deep learning models for building data-driven products. As deep learning models are fed with vast amounts of multi-modal data, often in the range of hundreds of millions of terabytes, embeddings – the internal representations of input data within these models – have become indispensable[5].

Embeddings are numerical representations of machine learning features used as inputs into deep learning models. One of the early works on word embeddings can be traced back to the classical neural language model[4] by Bengio et al. (2003) and the word2vec[25] model by Mikolov et al. (2013). The usage of embeddings to generate compressed, context-specific representations of content gained significant popularity after the publication of Google’s Word2Vec paper[25]. A text embedding is a projection of a piece of text into a high-dimensional latent space, where its position is represented by a vector, a long sequence of numbers, often with 768 or 1536 dimensions – akin to the two-dimensional Cartesian coordinates in algebra, but with more dimensions. Mathematically, an embedding space, or latent space, is defined as a manifold in which similar items are positioned closer to one another than less similar items. In this context, sentences that are semantically similar should have similar embedded vectors and, consequently, be closer together in the space.

Embeddings can be thought of as "**learned representations**", as the initial encoding of words or tokens is allowed to communicate and gather information in a data-dependent manner during the learning process. In contrast to the previously discussed fingerprints, which are not learned representations, embeddings facilitate information aggregation from a sequence in a data-dependent way, making them a valuable tool for molecular fingerprinting.

Furthermore, embeddings in a space can be used to calculate the distance between embeddings of similar entities using popular distance metrics such as **cosine similarity** or **Jaccard similarity scores**. The idea of an embedding space is to form a clustered space where similar entities, like molecules with similar physicochemical properties, end up closer together, enabling more effective analysis and prediction.

5.2 Analogy to Language Modelling

Contextualized word embeddings have emerged as a powerful technique for natural language processing tasks, addressing the limitations of traditional static word embeddings like Word2Vec[25]. These earlier approaches assigned fixed vector representations to words, failing to capture polysemous words with multiple meanings based on context. The advent of language models like BERT[7], developed by Google, has enabled the generation of dynamic embeddings that incorporate contextual information. These models leverage self-attention mechanisms to compute context-aware representations for words by attending to their relationships with other words in the sentence. Specifically, the self-attention mechanism computes attention scores that quantify the contribution of each word to the representation of a given word, allowing for disambiguation of polysemous terms. This approach bears analogies to modeling chemical sequences, where words can be viewed as molecular fragments such as chains, rings, and functional groups. Just as contextualized word embeddings capture the nuances of natural language, contextualized

embeddings for chemical fragments could potentially encode the intricate relationships and interactions within molecular structures, paving the way for advanced applications in chem informatics and drug discovery.

5.3 Scope of Encoder-Decoder Models

Embeddings, derived from encoder-decoder architectures, offer a versatile framework for various predictive tasks in the realm of molecular modeling and optimization. These embeddings facilitate the generation of informative latent representations that can be efficiently mapped to target outputs through decoding mechanisms. One notable application involves reaction prediction, where an encoder projects input data onto a well-defined latent space, and a decoder subsequently translates the encoded representation into the desired product molecules. Realizing the full potential of such predictive tasks hinges on the development of robust encoders and decoders capable of generating semantically rich embeddings and accurately decoding them, respectively.

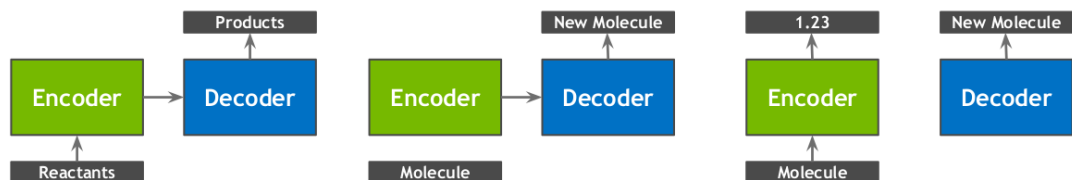


Figure 9: From left to right: Reaction Prediction, Molecular optimization, Property prediction, Property guided molecular generation

Moreover, encoder-decoder models hold promise for molecular optimization when appropriately fine-tuned. The encoder component alone can be leveraged to project molecules into a latent embedding space, enabling the extraction of meaningful features for downstream machine learning and deep learning algorithms. These embeddings can be utilized for property prediction tasks by training models on relevant property data. Furthermore, property-guided molecular generation becomes feasible when the embedding space exhibits well-defined clusters corresponding to specific molecular properties. By identifying suitable points within these clusters, desired molecules with targeted properties can be generated through the decoding process. Recent research by the NVIDIA CLARA[29] team has highlighted the potential of such embedding-based approaches in the field of molecular modeling and optimization.

6 Modeling

6.1 Input Representations: Tokenization

Tokenization is an essential pre-processing step for language models, as it transforms textual data into a sequence of numerical tokens amenable to computational processing. A common approach involves encoding text into a single, protracted list of integers via a lookup table, typically referred to as a vocabulary. Crucially, the quality of this vocabulary profoundly impacts the performance of language models, with more comprehensive vocabularies generally yielding superior results. In this regard, we adopted a methodology akin to that outlined in the GPT-2 paper[28] (Radford et al., 2019) under the "Input Representation" section.

Byte Pair Encoding[33] (BPE) represents a pragmatic compromise between character-level and word-level language modeling, effectively interpolating between the two by employing word-level inputs for frequent symbol sequences and character-level inputs for infrequent ones. Despite its name, contemporary BPE implementations often operate on Unicode code points rather than byte sequences, necessitating the inclusion of the entire Unicode symbol space to model all Unicode strings comprehensively. The typical BPE algorithm proceeds by initializing a character-level vocabulary, tallying the frequency of each possible symbol pair, and iteratively merging the most frequent pair into a new token until the desired vocabulary size is attained. Consequently, the initial merges yield tokens comprising two characters, with subsequent iterations generating progressively longer subwords.

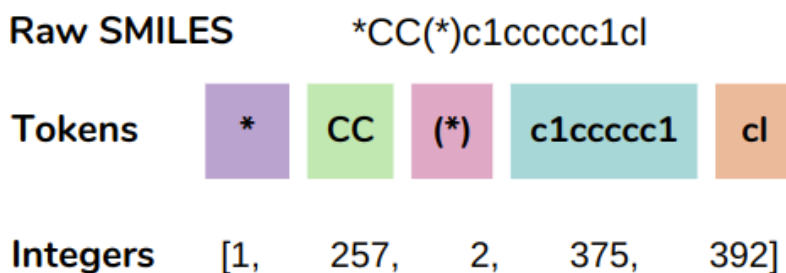
"Byte-Pair Encoding (BPE) was initially developed as an algorithm to compress texts, and then used by OpenAI for tokenization when pre-training the GPT model. It's used by a lot of Transformer models, including GPT, GPT-2, RoBERTa, BART, and DeBERTa."

6.2 Building Vocabulary

In addition to the Byte Pair Encoding (BPE) algorithm, several pre-processing stages were incorporated to identify special tokens and prevent bond dissolution during the identification of merges. Firstly, all necessary tokens were included for performing Masked Language Modeling and constructing batches of text delimited by special tokens, such as start and end of sequence markers. Secondly, a set of special tokens encompassed functional fragments[8] like carboxyl groups, hydroxyl groups, and benzene rings, along with miscellaneous tokens. Furthermore, all elements from the periodic table were included, except for frequently occurring elements like carbon, nitrogen, oxygen, fluorine, chlorine, and bromine. Excluding these elements mitigated the frequent fragmentation during pre-processing, which was a significant error in the PolyBERT[19] implementation that essentially resulted in character-level tokenization due to excessive segmentation in the pre-processing stage.

Table 4: Example vocabulary for Tokenizing Raw SMILES

Token	Token_id
*	1
(*)	2
CC	257
cl	392
c1ccccc1	375

**Figure 10:** Illustration on how to process Raw SMILES to tokens and token ids(integers) using a vocabulary

Moreover, a regular expression pattern was employed to customize the tokenization process by restricting certain merges. Specifically, it ensured that bonds were never split across different tokens, preventing incomplete bonds. This approach significantly improved the tokenization performance in unit tests. Code implementations are inspired from Andrej Karpathy's minbpe[16] GitHub repo.

"Actual vocabulary sizes of BERT and GPT-2 are 30,000 and 50,000 respectively"

Table 5: Design Choices for building our vocabulary

Token	Token_id
No of PSMILES trained on	100 Million (7.9GB .txt file)
vocabulary size	1000
Special Tokens	329
Mergers	648
Regex Pattern	"?[^-#:\s]*(?:[-#:] .){1}[])??"

Table 6: *Special Tokens Encoder Essentials*

Token	Token_name
"<pad>"	padding token
"<SOS>"	start of sequence
"<EOS>"	end of sequence
"<mask>"	masking token
"[UNK]"	unknown token
"*"	attachment site
"(*)"	attachment site 2
"(/*)"	attachment chiral site
"[*]"	attachment site square
"[/*]"	attachment chiral site square

Table 7: *Examples of Functional Groups as Special Tokens.*

Token	Token_name
"CC(F)(F)F"	1,1,1-trifluoroethane
"C1(C2=CC=CC=C2)=CC=CC=C1"	1,1'-biphenyl
"C1(CC=C2)=C2C=CC=C1"	1H-indene
"[NH]1CCCC1"	1H-pyrrole
"CC#CC"	2-butyne
"C(=O)Br"	acyl bromide
"CC=O"	aldehyde
"C(=O)N"	amide
"C1=CC=CC=C1"	benzene
"C(=O)O"	carboxylic acid
"COC"	dimethyl ether

Table 8: Vocabulary After Training Tokenizer using BPE

Token	Token_id
"<pad>"	0
"<SOS>"	1
"<EOS>"	2
"<mask>"	3
" [UNK] "	4
"*"	5
"(*)"	6
"(/*)"	7
"CC(F) (F)F"	8
"C1 (C2=CC=CC=C2)=CC=CC=C1"	9
"C1 (CC=C2)=C2C=CC=C1"	10
.	.
cc	371
CC	372
.	.
*)	998
nc7cc	999

```

1  from tokenizers import Tokenizer
2
3  #load tokenizer from file
4  t = Tokenizer.from_file("tokenizer_models/tokenizer-100m-HF-vocab1000.json")
5
6  # define psmile to test
7  psmile = '*CC(*) (C)C(=O)OCCN(CC)c1ccccc1'
8  #using encode function
9  out = t.encode(psmile)
10 smiles_tok = ""
11 for id in out.ids:
12     smiles_tok += f"{t.id_to_token(id)}" + ' '
13 print(f'Actual SMILES: {psmile} has length {len(psmile)}')
14 print(f'Toknized SMILES: {out.ids} has length{len(out.ids)}')
15 print(f'Id to Tokens: {smiles_tok}')
16 print(f'Compression ratio:{len(psmile)/len(out.ids)}')
17
18 # Results
19 Actual SMILES: *CC(*) (C)C(=O)OCCN(CC)c1ccccc1 has length 30
20 Toknized SMILES: [5, 372, 6, 18, 63, 400, 333, 372, 334, 536] has length10
21 Id to Tokens: *    CC    (*)    (C)    C(=O)OC    CN    (    CC    )    c1ccccc1
22 Compression ratio:3.0
23

```

Figure 11: Testing tokenizer trained on 100 million SMILES data. See GitHub for tokenizer training code.

6.3 Transformer Architecture

Transformer, a novel neural network architecture based on self-attention mechanism was introduced in Google's famous Neural Machine Translation paper "Attention is all you need"[35] in 2017.

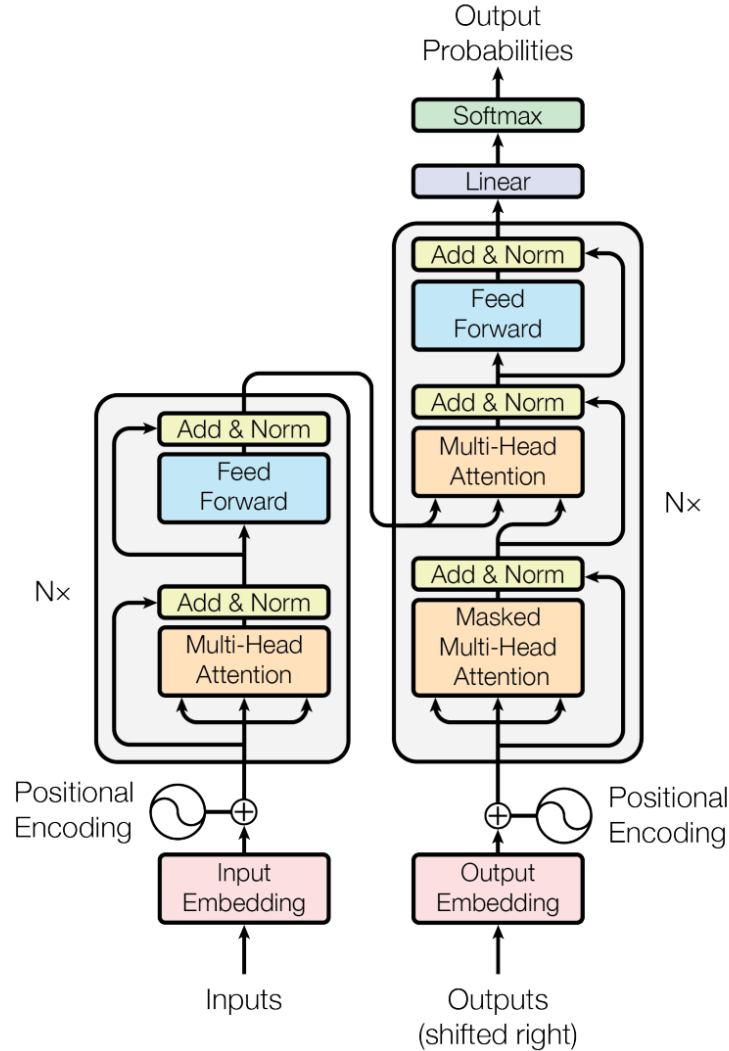


Figure 12: The Transformer's Architecture. Vaswani et al 2017

6.3.1 Attention

Attention is a communication mechanism initially developed for Neural Machine Translation problem. An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

6.3.2 Scaled Dot-Product Attention

The input consists of queries and keys of dimension d_k , and values of dimension d_v . We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values. In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q . The keys and values are also packed together into matrices K and V . We compute the matrix of outputs as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

6.3.3 Single Head of Attention

A single head of self-attention computes the attention scores using Scaled Dot-product attention as discussed above. Code structure adopted from Andrej Karpathy's minGPT[15]

```
1 import torch
2 import math
3 import torch.nn as nn
4 import torch.nn.functional as F
5
6
7 class Head(nn.Module):
8     """
9     One head on self - attention
10    """
11    def __init__(self, head_size, n_embd, dropout):
12        super().__init__()
13        self.key = nn.Linear(n_embd, head_size, bias = False)
14        self.query = nn.Linear(n_embd, head_size, bias = False)
15        self.value = nn.Linear(n_embd, head_size, bias = False)
16        self.dropout = nn.Dropout(dropout)
17
18    def forward(self, x):
19        B, T, C = x.shape
20        k = self.key(x) # B, T, head_size
21        q = self.query(x) # B, T, head_size
22
23        # compute the self attention scores
24        wei = q @ k.transpose(-2, -1) * C**-0.5 # (B, T, C) @ (B, C, T) --> (B, T, T)
25        wei = F.softmax(wei, dim=-1) # softmax
26        wei = self.dropout(wei)
27        # perform weighted aggregation
28        v = self.value(x) # B, T, C
29        out = wei @ v # (B,T,T) @ (B,T, C) ---> (B, T, C)
30        return out
```

Figure 13: Py-Torch implementation: class definition for single head of attention

6.3.4 Multi-Head Attention

Instead of performing a single head of attention, it employs h distinct learned linear projections to map the queries, keys, and values to sub dimensional spaces of d_k , d_k , and d_v , respectively. Parallel attention computations are then performed across these projected query, key, and value representations, yielding h output values of dimensionality d_v . This approach facilitates multiple concurrent information flow channels, enabling the attention mechanism to capture and propagate diverse facets of the input more effectively. The multi head attention architecture can be conceptualized as a ensemble of attention "heads", each specializing in distinct aspects of the input data, collectively synthesizing a more comprehensive representation.

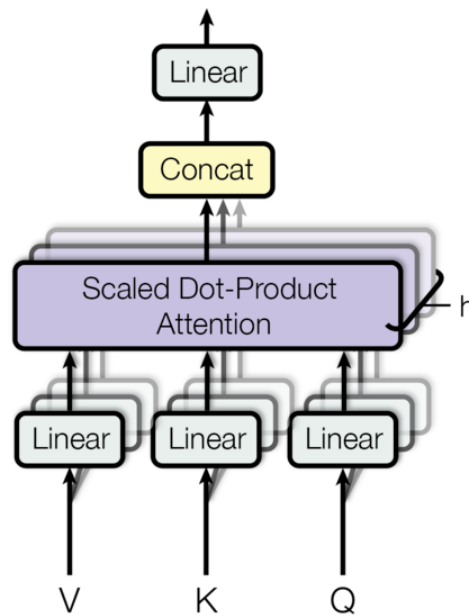


Figure 14: Multi-head Self-Attention in Transformer. Vaswani et al 2017

```

1 import torch
2 import math
3 import torch.nn as nn
4 import torch.nn.functional as F
5
6
7 class MultiHeadAttention(nn.Module):
8     """ multiple heads of self attention in parallel"""
9     def __init__(self, n_head, head_size, n_embd, dropout):
10         super().__init__()
11         self.heads = nn.ModuleList([Head(head_size=head_size, n_embd=n_embd,
12                                         ↪ dropout=dropout ) for _ in range(n_head)])
13         self.proj = nn.Linear(n_embd, n_embd)
14         self.dropout = nn.Dropout(dropout)
15
16     def forward(self, x):
17         out = torch.cat([h(x) for h in self.heads], dim=-1)
18         out = self.dropout(self.proj(out))
19         return out

```

Figure 15: *Py-Torch implementation: class definition for multi-head of attention*

6.3.5 Position-wise Feed-Forward Networks

In the encoder and decoder layers of the described model, a fully connected feed-forward network is incorporated in addition to the attention sub-layers. This feed-forward network comprises two linear transformations separated by a ReLU activation function. While the linear transformations are applied identically across all positions, their parameters vary across different layers. Alternatively, this component can be viewed as two convolutional operations with a kernel size of 1. The input and output dimensionality is set to $d_{model} = 768$, while the inner layer dimensionality is $d_{ff} = 3072$.

$$FFN(x) = \max(0, xW1 + b1)W2 + b2 \quad (2)$$

```

1 import torch
2 import math
3 import torch.nn as nn
4 import torch.nn.functional as F
5
6 class FeedForward(nn.Module):
7     """a simple linear layer followed by a non-linearity """
8
9     def __init__(self,n_embd,dropout):
10         super().__init__()
11         self.net = nn.Sequential(
12             nn.Linear(n_embd, 4*n_embd),
13             nn.ReLU(),
14             nn.Linear(4*n_embd, n_embd),
15             nn.Dropout(dropout),
16         )
17
18     def forward(self, x):
19         return self.net(x)

```

Figure 16: *Py-Torch implementation: class definition for Position-wise Feed Forward Network*

6.3.6 Encoder Block

The encoder block consist of two sub-layers, a self attention layer followed by a feed forward Neural network. The encoders are all identical in structure(yet they do not share weights).The encoder's inputs first flow through a self-attention layer – a layer that helps the encoder look at other words in the input sentence as it encodes a specific token. The outputs of the self-attention layer are fed to a feed-forward neural network. The exact same feed-forward network is independently applied to each position.

```

1 import torch
2 import math
3 import torch.nn as nn
4 import torch.nn.functional as F
5
6 class Block(nn.Module):
7     """ Trnaformer bloack : communication followed by computation"""
8     def __init__(self, n_embd, n_head, dropout):
9         super().__init__()
10         head_size = n_embd // n_head
11         self.sa = MultiHeadAttention(n_head = n_head, head_size=head_size, n_embd= n_embd,
12                                     ↪ dropout =dropout )
13         self.ffwd = FeedForward(n_embd=n_embd, dropout=dropout)
14         self.ln1 = nn.LayerNorm(n_embd)
15         self.ln2 = nn.LayerNorm(n_embd)
16
17     def forward(self, x):
18         x = x + self.sa(self.ln1(x))
19         x = x + self.ffwd(self.ln2(x))
20         return x

```

Figure 17: *Py-Torch implementation: class definition for Encode Block*

6.3.7 Embedding and Softmax

Similarly to other sequence transduction models, we use learned embeddings to convert the input tokens and output tokens to vectors of dimension d_{model} . We also use the usual learned linear transformation and softmax function to convert the decoder output to predicted next-token probabilities. In our model, we share the same weight matrix between the two embedding layers and the pre-softmax linear transformation. In the embedding layers, we multiply those weights by $\sqrt{d_{model}}$.

6.3.8 Positional Encoding

Attention has no notion of space. nodes have no information about their space. Thus Positional encoding is required before self attention.

$$\text{Embedding} = \text{Token embedding} + \text{Positional encoding} \quad (3)$$

6.4 Additional resources

- **The illustrated Transformer**[1] by Jay Alammar
- **The Annotated Transformer**[31] by Harvard NLP
- **Attention? Attention!**[38] by Lilian Weng
- **BERT Research series**[23] by Chris McCormick
- **Mastering BERT**[17] by Chee Kean

6.5 Language Model

```
1 import torch
2 import math
3 import torch.nn as nn
4 import torch.nn.functional as F
5
6 class BERTLanguageModel(L.LightningModule):
7
8     def __init__(self, vocab_size, n_embd, block_size, n_head,
9         ↪ n_layer, dropout, lr, weight_decay, betas, warmup_steps):
10         super().__init__()
11         self.lr = lr
12         self.weight_decay = weight_decay
13         self.betas = betas,
14         self.warmup_steps = warmup_steps
15
16         self.vocab_size = vocab_size
17         self.n_embd = n_embd
18         self.block_size = block_size
19
20         self.save_hyperparameters()
21         self.token_embedding_table = nn.Embedding(vocab_size, n_embd, padding_idx=0)
22         self.position_embedding_table = nn.Embedding(block_size, n_embd, padding_idx=0)
23         self.blocks = nn.Sequential(*[Block(n_embd=n_embd, n_head=n_head, dropout=dropout)
24             ↪ for _ in range(n_layer)])
25         self.ln_f = nn.LayerNorm(n_embd)
26         self.lm_head = nn.Linear(n_embd, vocab_size)
27
28         print("Total Parameters:", sum([p.nelement() for p in self.parameters()]))
29
30     def forward(self, idx, targets):
31         B, T = idx.shape
32
33         tok_embedding = self.token_embedding_table(idx) # BTC C- embed C
34         pos_embedding = self.position_embedding_table(torch.arange(T).to(self.device)) #
35             ↪ torch.arange(T) #(T,C)
36
37         x = tok_embedding + pos_embedding
38         x = self.blocks(x)
39         logits = self.lm_head(x) # BTC
40         return logits
```

Figure 18: *Py-Torch Lightning implementation: class definition for BERTLanguage Model*

6.6 Sentence-Transformer

Sentence embeddings[32] are vector representations that capture the semantics of entire sentences, analogous to how word embeddings represent individual words. Neural network models, such as BERT, are designed to process numerical data, and word embeddings provide a way to represent words as vectors. However, to fully interpret the meaning of a sentence, a pooling strategy is required at the output layer to aggregate the individual token embeddings.

In the context of this research, polymer sequences can be viewed as analogous to sentences, where the fragments within the sequence resemble words. The adopted pooling mechanism involves pooling over all tokens except for padding tokens, which are excluded through an attention mask that filters out the non-informative padded tokens.

By applying this pooling strategy to the output embeddings of each token from models like BERT, a single vector representation, known as a sentence embedding, is obtained. Just as word embeddings are vector representations of individual words, sentence embeddings are vector representations that encapsulate the semantics of an entire sentence or, in this case, a polymer sequence.

This approach allows for the interpretation of the sequence as a whole, rather than treating each token or fragment in isolation. The resulting sentence embeddings can then be utilized for various downstream tasks, such as similarity analysis, clustering, or prediction tasks, leveraging the rich semantic information captured within these vector representations.

6.7 Masked Language Modelling

Masked Language Modeling (MLM) is a pre-training strategy employed by models like BERT (Bidirectional Encoder Representations from Transformers). The fundamental concept involves masking a portion (typically 15%) of the tokens within a sequence and tasking the model to predict the masked tokens. Crucially, the model can attend to the surrounding tokens bidirectionally, leveraging both left and right contexts. This bidirectional attention mechanism enables the model to develop a comprehensive contextual understanding of the entire sequence, which proves invaluable for tasks requiring such holistic comprehension.

The training process for MLM involves separately storing the original labels for the masked tokens and computing the loss solely based on the model's ability to predict these masked elements accurately. By attending to the neighboring tokens in both directions, the model can effectively harness the contextual information to make informed predictions about the masked tokens. This pre-training approach endows models like BERT with robust language understanding capabilities, making them well-suited for a wide range of natural language processing tasks.

6.8 Pre-Training using Masked Language Modelling

Pretraining of the model is done in a batch-wise manner. Each batch has a dimension of (B, T), where B roughly speaking is the number of example sequences you want to include per batch, and T is the maximum context length (the maximum number of tokens per sequence). The training sequences are laid out as rows, delimited by special <|SOS|> and <|EOS|> tokens. One needs to be careful while tuning the maximum context length, as attention complexity is quadratic with context length, so it can quickly flood the GPU memory. One thumb rule for setting the context length is that you should set it to a value that works well for your data, and not to the limitations of the GPUs you have. If needed, you should upgrade your GPUs. Pre-training is a GPU compute-heavy process. Pre-training can be expensive and time-consuming, but once trained, the model is really powerful and fast. According to our settings, we had approximately 88 million parameters. The size of the model increases with parameters like the number of embedding dimensions, the number of heads, the number of layers, and the vocabulary size. The training code-base is written in PyTorch Lightning, enabling training on multi-node multi-GPU setups. We adopted a training strategy called **Distributed Data Parallel (DDP)** to train the model on multi-GPU. DistributedDataParallel (DDP) implements data parallelism at the module level, which can run across multiple machines. Essentially, the batches are distributed to all available GPUs, thus reducing the effective training batches and saving a lot of time.

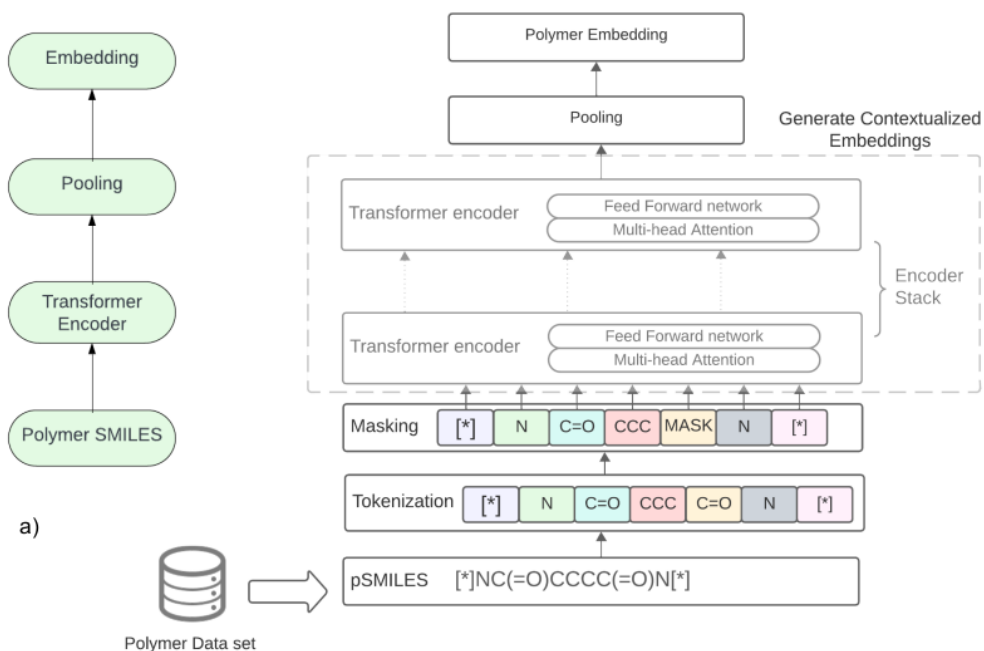


Figure 19: Flow diagram for generating embedding from pSMILES. Firstly a polymer sequence of variable length is fed into the model. The transformer model generates contextualized token embedding for all input fragments in the same order. A pooling layer eg: mean pooling averages the token embeddings from the model to get a fixed length vector. Finally a vector of a fixed length equal to the size of the embedding dimension is the output. b. The model tokenizes the pSMILES into meaningful fragments and masks certain tokens for Masked Language Modeling. The encoder stack then generates contextualized embeddings for each of the chemical fragments. For preparing the polymer embedding a pooling layer is introduced which captures and aggregates the essences of every token embedding.

Total number of trainable parameters = 88.7 Million

Table 9: *Model Hyper parameters*

Hyper parameter	Description	Value
Vocab_size	size of vocabulary	1000
Block_size	maximum context length	512
Batch_size	no of examples per batch	64
n_embd	Out put embedding dimension	768
n_head	no of heads in multihead	12
dropout	dropout for layers	0.2

Table 10: *Hyper parameters and design choices for trainer*

Hyper parameter	Value
Optimizer	Adam
Learning_rate	1e-4
Weight_decay	0.01
Warmup_steps	1000
Strategy	Distributed Data Parallel

7 Results and Discussions

After pre-training, in order to estimate the quality of the embeddings we used the embeddings of BERT-PolyLang to predict property(Tg Glass Transition temperature) on a data set than contain 7k molecules using a train test split of 80:20. We compared the quality of property prediction using r2_score of various other finger prints and also the current state-of-the-art PolyBERT. For all the finger prints we used same Deep learning model, simple Deep Neural Network with same hyper parameters to keep everything fair.

We observe that language-based learned embeddings performed better compared to traditional hand crafted fingerprints like One Hot Encoding and Morgan Finger print.

Table 11: RMSE and r2 score for property prediction for 7000 polymer monomers using different fingerprinting techniques, Model 1 : trained on 10K sequences, Model 2 trained on 100 K sequences

Fingerprint	Type of fingerprint	r2_score	RMSE
PolyBERT	Language-based:Learned	0.88	38.15
Morgan Fingerprint	Hand crafted	0.87	38.658
PolyLang	Language-based:Learned	0.81	48.37
One Hot Encoding	Hand crafted	0.76	53.90

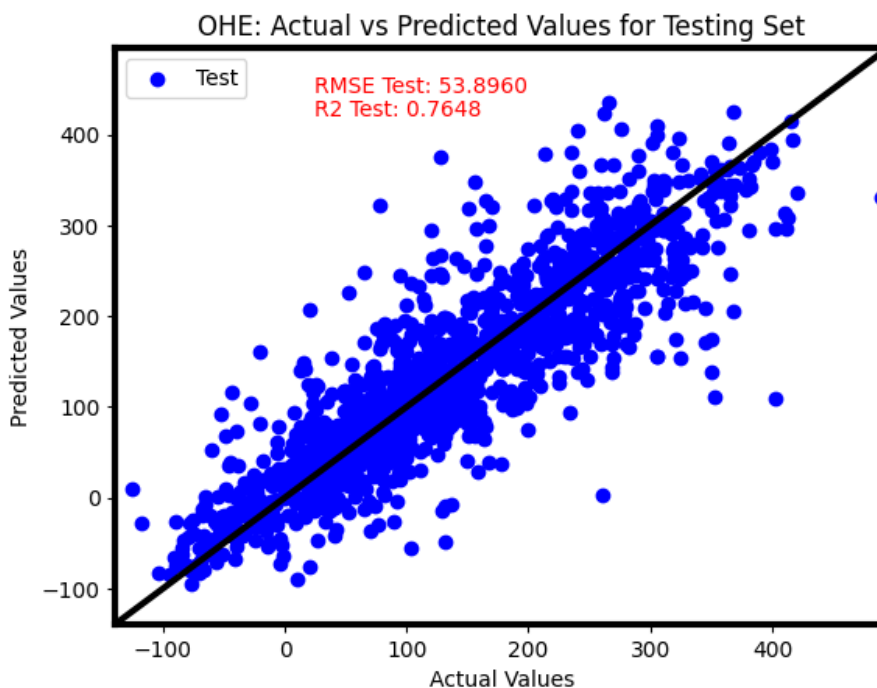


Figure 20: Actual vs Predicted values on test set for OHE

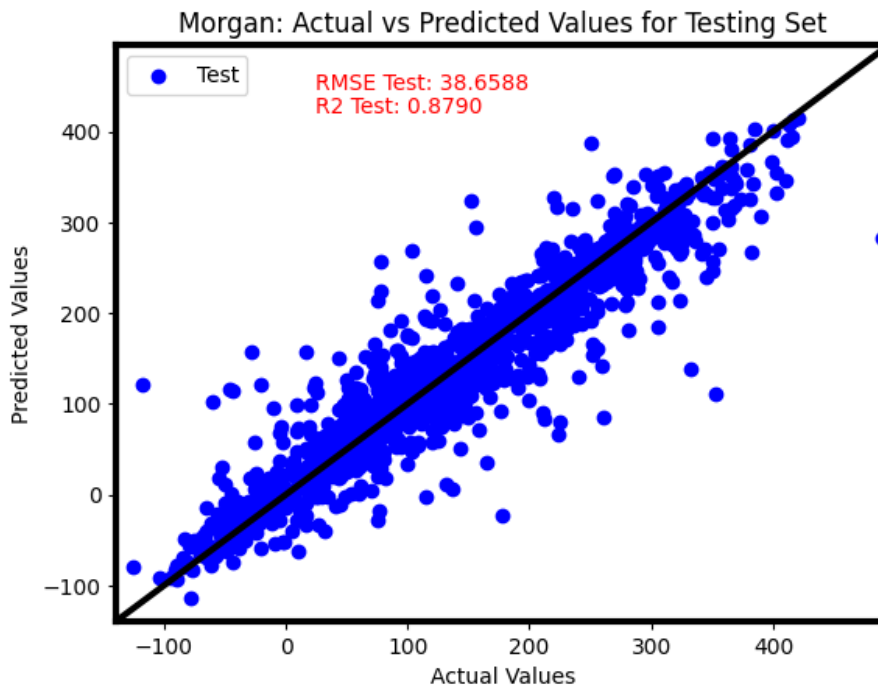


Figure 21: *Actual vs Predicted values on test set for Morgan*

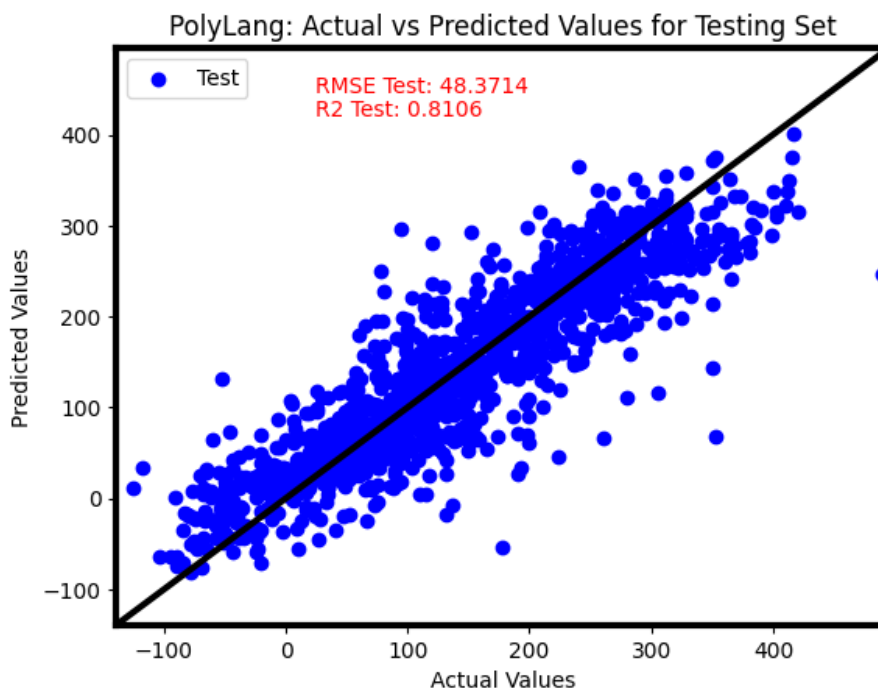


Figure 22: *Actual vs Predicted values on test set for PolyLang*

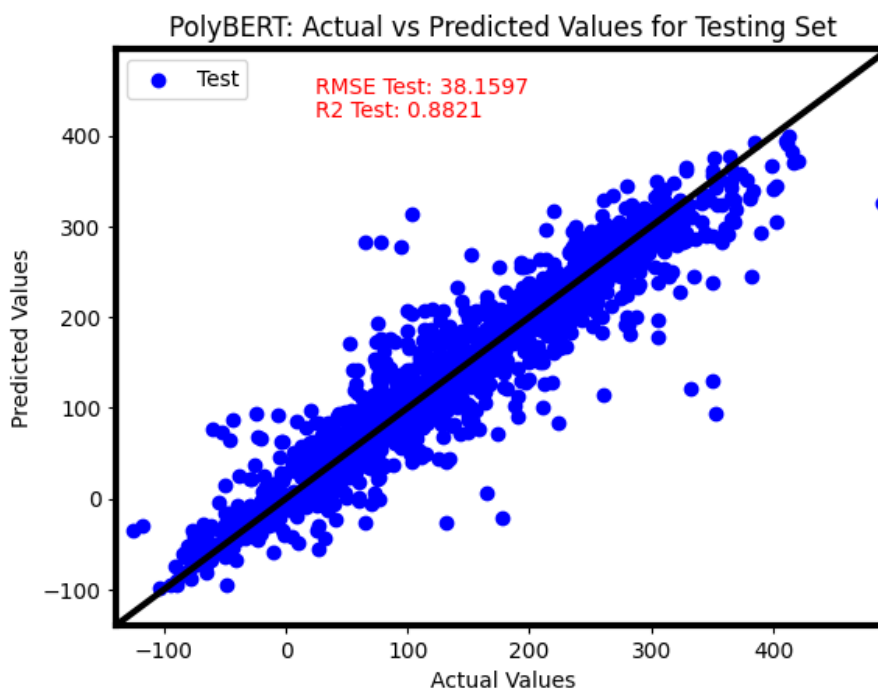


Figure 23: Actual vs Predicted values on test set for PolyBERT

We also experimented on how the performance is increasing with increasing number of training examples. Even with 1000 sequences, PolyLang achieved a r_2 score of 0.65. The model performance is significantly improving with number of training sequences it gets trained on.

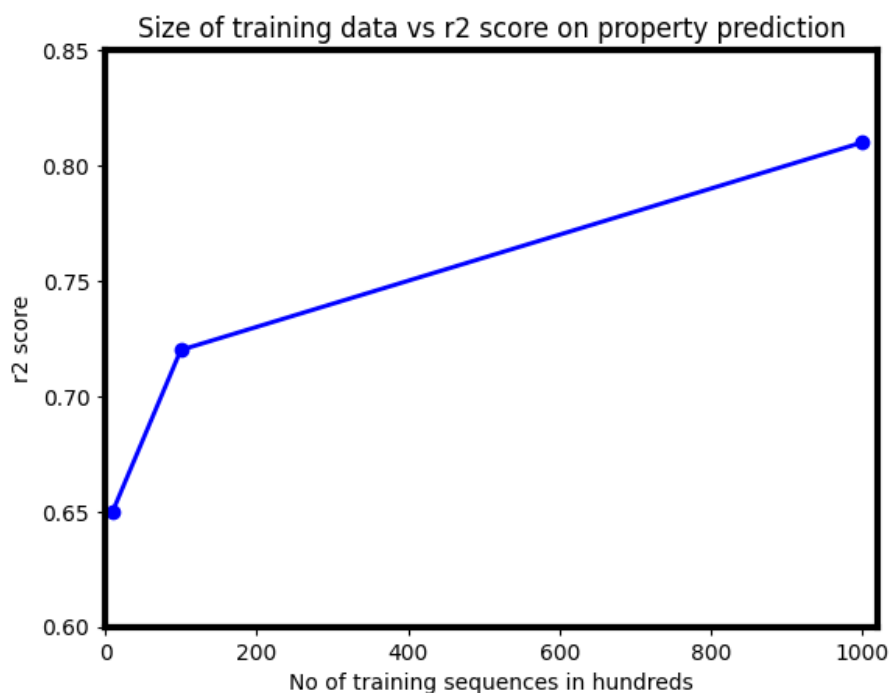


Figure 24: Embedding quality improvement with increasing training data: Size of training data Vs r_2 score for property (glass transition temperature) prediction for 7000 polymer monomers

To better understand these molecular representations, we performed Uniform Manifold Approximation and projection (UMAP)[24] on all fingerprints. UMAP is a non linear dimensionality reduction technique that enables the visualization of high dimensional data in 2D representations. Figure show the projection of the molecular fingerprints in a 2D map using UMAP technique. Among the three maps, we can observe that Morgan produced the better spread and unique space. The clusters formed are well defined and informative with respect to the property values indicated (Tg values on the scale). PolyLang also produces a latent space which spread and unique, we expect more clarity in latent space with more training data.

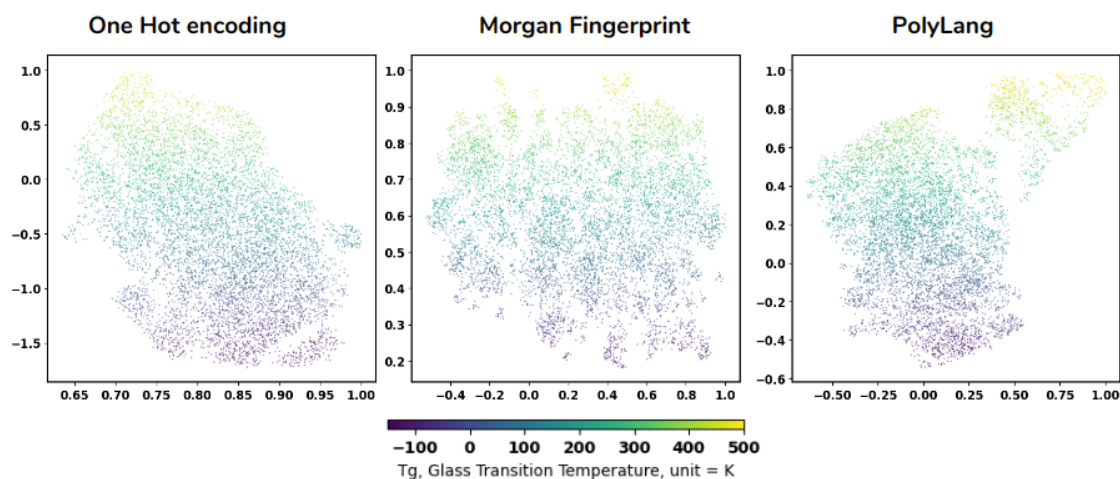


Figure 25: The 2D UMAP pot of molecular finger prints. Left to right, One Hot Encoding, Morgan Fingerprint, PolyLang

8 Conclusions and Future Work

Language model-based embeddings have emerged as a promising approach for learning better representations from SMILES (Simplified Molecular-Input Line-Entry System) data in the field of language modeling. Our research has demonstrated that the quality of embeddings improves with access to larger and more diverse training datasets. Language model representations have outperformed traditional hand-crafted methods, highlighting the effectiveness of data-driven techniques. The tokenization approach is a fundamental aspect of any language model, and significant performance differences have been observed between character-level and chunk-level models. Naive attention mechanisms have proven to be suitable for simple vocabularies, while more advanced techniques like flash, sparse, or dilated attention typically work well for larger contexts in natural language processing tasks.

We have found that pre-training with minimal design and better vocabulary selection is optimal for simple languages. Developing code from scratch using Python and PyTorch has enabled full customization. By far we have built code base for language model pre-training based on masked language modeling. This code base can be trained on any number of sequences with multi-GPU setups. To further advance the field, we have collected 200 million sequences for training, and with sufficient GPU computing resources, they aim to train the largest chemical language model with state-of-the-art performance.

Future work in this area may involve extending the encoder-only model to an encoder-decoder model, similar to the BART (Bidirectional and Auto-Regressive Transformers) model, unlocking the potential for property-guided molecular generation. Additionally, we are exploring the development of models that can learn an informative and clustered latent space, as clustered spaces lead to controllable generation, which is a desirable outcome in molecular design and optimization.

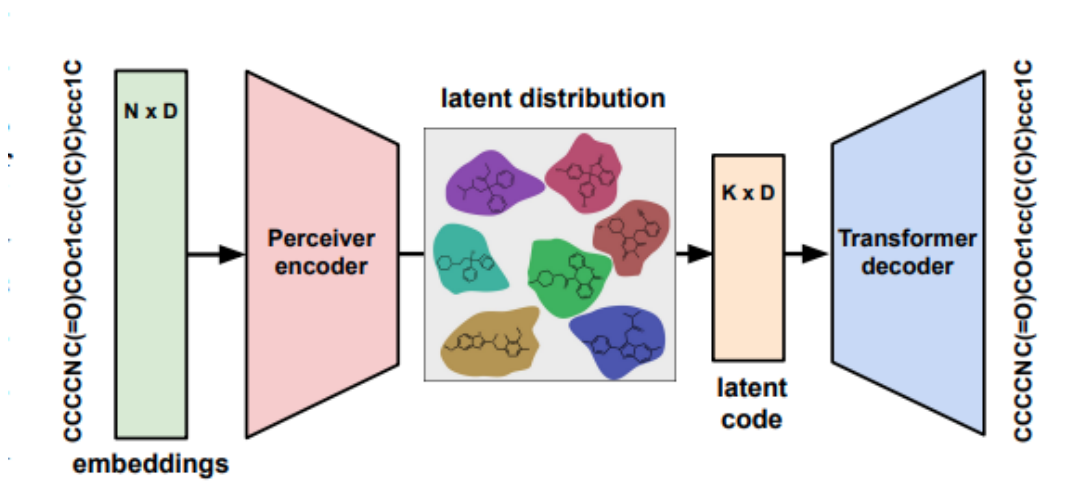


Figure 26: Creation of informative and clustered latent space using encoder -decoder model

References

1. Jay Alammar. The illustrated transformer, 2020. URL <http://jalamar.github.io/illustrated-transformer/>.
2. Debra J. Audus and Juan J. de Pablo. Polymer informatics: Opportunities and challenges. *ACS Macro Letters*, 6(10):1078–1082, 2017. doi: 10.1021/acsmacrolett.7b00228. URL <https://doi.org/10.1021/acsmacrolett.7b00228>. PMID: 29201535.
3. Agrim Babbar, Sriram Ragunathan, Debirupa Mitra, Arnab Dutta, and Tarak K. Patra. Explainability and extrapolation of machine learning models for predicting the glass transition temperature of polymers. *Journal of Polymer Science*, 62(6):1175–1186, 2024. doi: <https://doi.org/10.1002/pol.20230714>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/pol.20230714>.
4. Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Janvin. A neural probabilistic language model. *J. Mach. Learn. Res.*, 3(null):1137–1155, mar 2003. ISSN 1532-4435.
5. Vicki Boykis. What are embeddings, 2024. URL https://github.com/veekaybee/what_are_embeddings.
6. Jörg Degen, Christof Wegscheid-Gerlach, Andrea Zaliani, and Matthias Rarey. On the art of compiling and using ‘drug-like’ chemical fragment spaces. *ChemMedChem*, 3(10):1503–1507, 2008. doi: <https://doi.org/10.1002/cmdc.200800178>. URL <https://chemistry-europe.onlinelibrary.wiley.com/doi/abs/10.1002/cmdc.200800178>.
7. Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
8. global chem. Open smiles, 2022. URL https://github.com/Sulstice/global-chem/blob/development/global_chem/global_chem/miscellaneous/open_smiles.py.
9. Garrett B. Goh, Nathan O. Hodas, Charles Siegel, and Abhinav Vishnu. Smiles2vec: An interpretable general-purpose deep neural network for predicting chemical properties, 2018.
10. Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. Deberta: Decoding-enhanced bert with disentangled attention, 2021.
11. Maya Hirohara, Yutaka Saito, Yuki Koda, Kengo Sato, and Yasubumi Sakakibara. Convolutional neural network based on smiles representation of compounds for detecting chemical motif. *BMC Bioinformatics*, 19, 2018. URL <https://api.semanticscholar.org/CorpusID:57189630>.
12. Tran Doan Huan, Arun Mannodi-Kanakkithodi, and Rampi Ramprasad. Accelerated materials property predictions and design using motif-based fingerprints. *Physical Review B*, 92(1), July 2015. ISSN 1550-235X. doi: 10.1103/physrevb.92.014106. URL <http://dx.doi.org/10.1103/PhysRevB.92.014106>.
13. DAYLIGHT Chemical Information Systems Inc. 4. smarts - a language for describing molecular patterns, 2007. URL <https://www.daylight.com/dayhtml/doc/theory/theory.smarts.html>.

14. DAYLIGHT Chemical Information Systems Inc. Smiles - a simplified chemical language, Feb 2008. URL <https://www.daylight.com/dayhtml/doc/theory/theory.smiles.html>.
15. Andrej Karpathy. mingpt, 2023. URL <https://github.com/karpathy/minGPT/tree/master>.
16. Andrej Karpathy. minbpe, 2024. URL <https://github.com/karpathy/minbpe>.
17. Chee Kean. Mastering bert model: Building it from scratch with pytorch, 2022. URL <https://medium.com/data-and-beyond/complete-guide-to-building-bert-model-from-scratch-3e6562228891>.
18. Chiho Kim, Anand Chandrasekaran, Tran Doan Huan, Deya Das, and Rampi Ramprasad. Polymer genome: A data-powered polymer informatics platform for property predictions. *The Journal of Physical Chemistry C*, 122(31):17575–17585, 2018. doi: 10.1021/acs.jpcc.8b02913. URL <https://doi.org/10.1021/acs.jpcc.8b02913>.
19. Christopher Kuenneth and Rampi Ramprasad. polybert: a chemical language model to enable fully machine-driven ultrafast polymer informatics. *Nature Communications*, 14(1), July 2023. ISSN 2041-1723. doi: 10.1038/s41467-023-39868-6. URL <http://dx.doi.org/10.1038/s41467-023-39868-6>.
20. Tzyy-Shyang Lin, Connor W. Coley, Hidenobu Mochigase, Haley K. Beech, Wencong Wang, Zi Wang, Eliot Woods, Stephen L. Craig, Jeremiah A. Johnson, Julia A. Kalow, Klavs F. Jensen, and Bradley D. Olsen. Bigsmiles: A structurally-based line notation for describing macromolecules. *ACS Central Science*, 5(9):1523–1531, 2019. doi: 10.1021/acscentsci.9b00476. URL <https://doi.org/10.1021/acscentsci.9b00476>. PMID: 31572779.
21. Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach, 2019.
22. Ruimin Ma and Tengfei Luo. P11m: A benchmark database for polymer informatics. *Journal of Chemical Information and Modeling*, 60(10):4684–4690, 2020. doi: 10.1021/acs.jcim.0c00726. URL <https://doi.org/10.1021/acs.jcim.0c00726>. PMID: 32986418.
23. Chris McCormick. Bert research series, 2019. URL <https://mccormickml.com/2019/11/11/bert-research-ep-1-key-concepts-and-sources/>.
24. Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction, 2020.
25. Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space, 2013.
26. NVIDIA. Megamolbart, Nov 2023. URL <https://catalog.ngc.nvidia.com/orgs/nvidia/teams/clara/models/megamolbart>.
27. Shingo Otsuka, Isao Kuwajima, Junko Hosoya, Yibin Xu, and Masayoshi Yamazaki. Polyinfo: Polymer database for polymeric materials design. In *2011 International Conference on Emerging Intelligent Data and Web Technologies*, pages 22–29, 2011. doi: 10.1109/EIDWT.2011.13.

28. Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019. URL <https://api.semanticscholar.org/CorpusID:160025533>.
29. Danny Reidenbach, Micha Livne, Rajesh K. Ilango, Michelle Gill, and Johnny Israeli. Improving small molecule generation using mutual information machine. In *Proceedings of MLDD workshop, ICLR 2023*, 2014.
30. David Rogers and Mathew Hahn. Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50(5):742–754, 2010. doi: 10.1021/ci100050t. URL <https://doi.org/10.1021/ci100050t>. PMID: 20426451.
31. Sasha Rush. The annotated transformer, 2022. URL <https://nlp.seas.harvard.edu/annotated-transformer/>.
32. Omar Sanseviero. Sentence embeddings. introduction to sentence embeddings, 2024. URL https://osanseviero.github.io/hackerllama/blog/posts/sentence_embeddings/. Accessed on April 10, 2024.
33. Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units, 2016.
34. Haripriyan Uthayakumar, Rahul Krishna K, Raj Jain, Rajnish Kumar, and Tarak K. Patra. Qrchem: A deep learning framework for materials property prediction and design using qr codes. *ACS Engineering Au*, 4(1):91–98, 2024. doi: 10.1021/acsengineeringau.3c00055. URL <https://doi.org/10.1021/acsengineeringau.3c00055>.
35. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.
36. Sheng Wang, Yuzhi Guo, Yuhong Wang, Hongmao Sun, and Junzhou Huang. Smilesbert: Large scale unsupervised pre-training for molecular property prediction. BCB ’19, page 429–436, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450366663. doi: 10.1145/3307339.3342186. URL <https://doi.org/10.1145/3307339.3342186>.
37. David Weininger. Smiles, a chemical language and information system. 1. introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36, 1988. doi: 10.1021/ci00057a005. URL <https://doi.org/10.1021/ci00057a005>.
38. Lilian Weng. Attention? attention!, 2018. URL <https://lilianweng.github.io/posts/2018-06-24-attention/>.
39. Changwen Xu, Yuyang Wang, and Amir Barati Farimani. Transpolymer: a transformer-based language model for polymer property predictions. *npj Computational Materials*, 9(1), April 2023. ISSN 2057-3960. doi: 10.1038/s41524-023-01016-5. URL <http://dx.doi.org/10.1038/s41524-023-01016-5>.